

# A Voyage to the Kernel



## Part 14

### Segment 3.3: Day 13

Last time we discussed the various types of operations and system calls. In this article, we will focus more on the literature. Like I mentioned, there are two different modes: the kernel and the user mode. Let's look at the two types of switching. The first is when you make a system call. After calling it, the task will go for codes that are operational in the kernel mode. The other case is when you deal with interrupt requests (IRQs). Soon after an IRQ, a handler is called and the control goes back to the task that was interrupted.

A system call may be used when you want to access a particular I/O device or file, or when you need to get privileged information. It may also be used when you require to execute a command or to change the execution context.

Now let me elucidate the whole process that governs an IRQ event. We'll assume that a particular process is running. An IRQ may occur while the process is running. Then, the task will be interrupted to call the corresponding interrupt handler and it is executed right there. In the next step, as mentioned before, the control goes back to the task (which is running in user mode) and the process is back to its original state.

Advanced users can comprehend the mode of initiation by looking at the code below:

```
typedef irqreturn_t (*irq_handler_t)(int, void *);
```

```
struct irqaction {
    irq_handler_t handler;
    unsigned long flags;
    cpumask_t mask;
    const char *name;
    void *dev_id;
    struct irqaction *next;
    int irq;
    struct proc_dir_entry *dir;
};

extern irqreturn_t no_action(int cpl, void *dev_id);
extern int __must_check request_irq(unsigned int, irq_
handler_t handler,
    unsigned long, const char *, void *);
extern void free_irq(unsigned int, void *);

struct device;

extern int __must_check devm_request_irq(struct device *dev,
unsigned int irq,
    irq_handler_t handler, unsigned long irqflags,
    const char *devname, void *dev_id);
extern void devm_free_irq(struct device *dev, unsigned int
irq, void *dev_id);
```

#### Non-free elements in the kernel

In an earlier column, I had discussed the non-free code portions in the kernel. And I received a number of queries regarding the subject. So, I think it is appropriate to discuss it here.

It is true that the kernel (from the original